
Paradigmas da Programação

Trabalho Prático — Cross-Platform Qt Game

PARK OUT

Gonçalo Marques — N° 30012478

Gonçalo Pardelhas — N° 30014665

5 de Junho de 2026

1. Introdução

O Park Out é um jogo de puzzle estratégico desenvolvido no âmbito da unidade curricular de Paradigmas da Programação. O jogador gere um parque de autocarros numa grelha 8×8: cada autocarro tem cor e direção fixas, e o objetivo é libertar os caminhos para que todos os passageiros embarquem no autocarro da sua cor com o menor número de movimentos possível.

A aplicação foi desenvolvida em C++ com Qt 6 / QML, compilável para Desktop e Mobile, aplicando de forma integrada os três paradigmas obrigatórios: Programação Orientada a Objetos, programação com estilo funcional e programação concorrente/assíncrona.

2. Arquitetura

2.1. Separação UI / Lógica

A separação lógica / UI é garantida pela fronteira QML / C++. O GameController é exposto ao motor QML via contexto raiz, disponibilizando propriedades reativas (Q_PROPERTY) e métodos invocáveis (Q_INVOKABLE). A interface nunca acede diretamente ao Board ou ao Bus: toda a comunicação é feita através de sinais e slots.

2.2. Organização por Ficheiro

Ficheiro	Responsabilidade
gamecontroller.h/.cpp	Orquestração do jogo e ponte QML ↔ C++
board.h/.cpp	Tabuleiro, slots de embarque e deteção de colisões
bus.h/.cpp	Entidade autocarro (cor, capacidade, direção, posição)
passenger.h	Entidade passageiro e serialização para QML
gameanalytics.h	Módulo funcional puro: pontuação e nível de perigo
persistence.h/.cpp	Persistência local (JSON) de pontuações e progresso
gametests.h	Testes unitários da lógica principal
levels.json	Definição declarativa dos níveis (embebido como recurso Qt)
Main.qml	Interface gráfica completa (menu, tabuleiro, HUD, animações)

3. Motor de Jogo e Regras

3.1. Tabuleiro e Autocarros

O tabuleiro é uma grelha 8×8. Cada autocarro possui cor, capacidade (4, 6, 8 ou 12 lugares), direção fixa (Up, Down, Left, Right) e posição. O tamanho visual é proporcional à capacidade: comprimento = capacidade / 2 células. Um autocarro de capacidade 12 ocupa assim 6 células, tornando-o visualmente distinto no tabuleiro.

A verificação de obstrução percorre todos os autocarros, calculando as células reais que cada um ocupa com base na sua direção e comprimento, excluindo o próprio autocarro através do parâmetro ignoreBusIndex.

3.2. Movimento Passo-a-Passo

Ao clicar num autocarro, o handleBusClick calcula o destino final com um ciclo iterativo: avança a posição candidata célula a célula até encontrar um obstáculo ou a borda do tabuleiro. O movimento é então executado passo-a-passo via QTimer, emitindo dataChanged() a cada passo para atualizar a UI de forma fluida. O contador de jogadas é incrementado uma única vez no final do movimento completo.

3.3. Embarque de Passageiros

Quando um autocarro atinge a borda do tabuleiro e existe um slot livre, fica estacionado numa plataforma. O processo de embarque inicia-se automaticamente via processPassengerBoarding(): o próximo passageiro da fila é comparado com os autocarros estacionados por cor. O embarque ocorre passageiro a passageiro com animação. Cada passageiro só pode embarcar no autocarro da sua cor; ao atingir a capacidade máxima, o autocarro é removido da plataforma e o slot fica disponível.

3.4. Condições de Vitória e Derrota

Estado	Condição
Vitória	Tabuleiro sem autocarros + fila de passageiros vazia + plataformas vazias
Derrota	Sem slots livres E o próximo passageiro não tem autocarro compatível estacionado disponível

3.5. Sistema de Pontuação e Tempo

A pontuação é calculada pela fórmula: $\text{Score} = (\text{passageiros_embarcados} \times 100) - (\text{número_de_jogadas} \times 5)$, com mínimo de zero. O temporizador por nível conta o tempo decorrido em segundos. Ambos — melhor pontuação e melhor tempo — são persistidos localmente e apresentados no ecrã de seleção de níveis.

4. Níveis e Persistência

4.1. Definição dos Níveis

Os três níveis obrigatórios estão definidos em `levels.json`, inserido como recurso Qt (`qt_add_resources`). Cada nível especifica: identificador, número de slots (entre 4 e 8), sequência ordenada de passageiros por cor e lista de autocarros com cor, capacidade, direção e posição inicial.

Nível	Slots	Autocarros	Passageiros	Cores	Dificuldade
1	4	4	22	Red, Blue, Green, Yellow	Introdutório
2	5	8	42	Orange, Red, Blue, Green	Intermédio
3	6	8	56	Red, Blue, Green, Yellow	Avançado

4.2. Persistência Local

A classe `Persistence` gere um ficheiro `savegame.json` no diretório de dados local da aplicação, garantindo compatibilidade cross-platform. Por nível são registados três valores: melhor pontuação, melhor tempo em segundos e estado de conclusão. A escrita ocorre apenas se o novo valor for melhor que o registado, evitando regressão de recordes.

5. Programação Orientada a Objetos

A arquitetura segue os princípios da POO com encapsulamento rigoroso e responsabilidade única por classe. A herança é usada apenas onde semanticamente justificada: `GameController` herda de `QObject` para integração com o sistema de sinais/slots do Qt.

Classe	Responsabilidade	Padrão / Nota
Bus	Estado e comportamento de um autocarro	Value type com getters/setters
Board	Tabuleiro, slots e deteção de colisões	Composição com <code>std::vector<Bus></code>
GameController	Orquestração do jogo e ponte QML ↔ C++	QObject com sinais e <code>Q_PROPERTY</code>
GameAnalytics	Cálculo de pontuação e perigo do tabuleiro	Static-only, sem estado
Persistence	Leitura/escrita de progresso local em JSON	Static-only, sem estado
Passenger	Passageiro com cor e serialização para QML	Struct leve com <code>toVariantMap()</code>

O encapsulamento é garantido por membros privados com getters/setters explícitos em `Bus` e `Board`. O `GameController` expõe ao QML apenas o mínimo necessário via `Q_PROPERTY`, mantendo o estado interno protegido. A `Board` disponibiliza `getBuses()` `const` para leitura e `getBusesMutable()` para escrita, evitando o anti-padrão `const_cast`.

6. Paradigma Funcional

O módulo `GameAnalytics` implementa o paradigma funcional com três funções puras estáticas, sem efeitos secundários e sem acesso a estado mutável externo. Qualquer instância do compilador com os mesmos argumentos produz sempre o mesmo resultado.

`calculateScore(moveCount, initialPassengers, remainingPassengers)`

Calcula a pontuação: cada passageiro embarcado vale 100 pontos e cada jogada desconta 5. O resultado nunca é negativo. Exemplo: 10 passageiros embarcados em 5 jogadas = 975 pontos.

`evaluateBoardDanger(activeBuses, freeSlots, queueSize)`

Classifica o estado do tabuleiro em quatro níveis de perigo com base numa heurística ponderada: `ESTAVEL` (perigo < 40), `MODERADO` (40–74), `PERIGO` (>= 75) e `CRITICO` (sem slots livres com autocarros ativos). Completamente determinista e testável de forma isolada.

`applyMove(snapshot, busIndex, deltaRow, deltaCol)` — Estruturas Imutáveis

Recebe um `BoardSnapshot` por valor e devolve um novo snapshot com a posição do autocarro atualizada e o `moveCount` incrementado. O snapshot original não é modificado, demonstrando semântica imutável em C++ por cópia de valor. Esta estrutura serve de base para uma futura implementação de Undo via histórico de estados, e é demonstrada em runtime pelo método `testImmutableExample()` do `GameController`.

7. Concorrência e Assincronia

7.1. Carregamento Assíncrono de Níveis

O carregamento dos níveis é realizado de forma assíncrona com `QtConcurrent::run()` e `QFutureWatcher`. O método `loadLevelAsync()` lança o parsing do JSON numa thread secundária do thread pool gerido pelo Qt, sem bloquear a thread principal da UI. O `QFutureWatcher` monitoriza a conclusão e invoca `applyLoadedLevel()` na thread principal, garantindo thread safety sem mutex explícito.

A escolha de `QtConcurrent` em vez de `QThread` explícita deveu-se à simplicidade da tarefa: execução única, sem comunicação bidirecional contínua. Os dados são produzidos na thread secundária numa struct `LevelData` simples (sem objetos Qt não thread-safe) e consumidos apenas após a conclusão, na thread principal.

7.2. Animações com QTimer

Mecanismo	Utilização	Justificação
QtConcurrent::run	Leitura e parsing de levels.json	Isola I/O de disco da thread UI
QFutureWatcher	Callback ao completar o carregamento	Entrega thread-safe ao event loop principal
QTimer (moveStepTimer, 150 ms)	Movimento passo-a-passo dos autocarros	Animação fluida sem thread extra
QTimer (boardingTimer, 300 ms)	Embarque animado de passageiros	Visibilidade do processo, sem bloquear
QTimer (timer, 1 s)	Cronómetro do nível	Contagem via sinal elapsedSecondsChanged

8. Interface Gráfica (UI/UX)

8.1. Estrutura QML

A interface em Main.qml está organizada em dois ecrãs com transição animada (opacity + translateY, 300 ms): menu de seleção de níveis e ecrã de jogo. O ecrã de jogo usa Flickable para suporte a scroll em dispositivos móveis com ecrã pequeno.

8.2. Adaptação Multi-Plataforma

O tamanho do tabuleiro adapta-se ao ecrã: $\text{boardSize} = \text{Math.min}(\text{width}, 520) - 16$. As células têm dimensão dinâmica $\text{cellSize} = \text{boardSize} / \text{cols}$, garantindo que o jogo é jogável sem scroll horizontal em qualquer resolução. Os autocarros suportam interação por toque e por rato através de um MouseArea com deteção de swipe (threshold de 15 px) e toque simples, sem código específico por plataforma.

8.3. Elementos da Interface

- Menu de seleção: três botões de nível com melhor pontuação, melhor tempo e estado de conclusão.
 - HUD: jogadas, pontuação, temporizador e indicador de perigo do tabuleiro em tempo real.
 - Fila de passageiros: círculos coloridos com animação de pulsação sinusoidal no primeiro da fila.
 - Plataformas de embarque: slots com sobreposição dos autocarros estacionados e rácio passageiros/capacidade.
 - Tabuleiro: grelha com autocarros coloridos, indicador de direção e ocupação atual/máxima.
 - Notificações: popup laranja temporário para caminho bloqueado, plataformas cheias, estacionamento.
-

-
- Overlay de fim de jogo: sobreposição verde (vitória) ou vermelha (derrota) com animação de escala e botão de ação.

9. Testes e Qualidade de Código

Os testes unitários estão implementados em `GameTests` e são executados automaticamente no arranque da aplicação via `GameTests::runAllTests()` em `main.cpp`. Utilizam `assert()` da biblioteca padrão e produzem output detalhado na consola. O módulo `GameAnalytics` é testável de forma completamente isolada por não depender de estado global.

Suite 1 — `GameAnalytics` (funções puras)

- `calculateScore(5, 10, 0) == 975`: $10 \times 100 - 5 \times 5 = 975$.
- `calculateScore(1000, 1, 1) == 0`: pontuação nunca negativa.
- `evaluateBoardDanger(3, 0, 5) == "CRITICO"`: sem slots livres com autocarros ativos.
- `evaluateBoardDanger(0, 6, 0) == "ESTAVEL"`: tabuleiro completamente livre.

Suite 2 — `Bus`

- Construção e getters (cor, capacidade, passageiros iniciais a 0, `isFull() == false`).
- Adição de 4 passageiros a um autocarro de capacidade 4 e verificação de `isFull() == true`.
- `setPosition()` e verificação das novas coordenadas.

Suite 3 — `Board`

- Dimensões 8×8, número de slots padrão (6), `hasFreeSlot()` e `getNextFreeSlotIndex()`.
 - Ocupação de todos os slots e verificação de `hasFreeSlot() == false`.
 - `clearSlots()` e verificação de restauro do estado livre.
 - `isOccupied()` para autocarro horizontal (Right, capacity 4): ocupa exatamente (r,c) e (r,c+1).
 - `isOccupied()` para autocarro vertical (Down, capacity 6): ocupa exatamente (r,c), (r+1,c) e (r+2,c).
 - `getBusesMutable()`: verificação do número de autocarros sem crash após adição.
-

10. Execução Multi-Plataforma

Plataforma	SO / Dispositivo	Qt Version	Toolchain	Kit
Desktop	Ubuntu 24.04 / Windows 11	Qt 6.x	GCC 13 / MSVC 2022	Desktop
Mobile	Android 13+	Qt 6.x	Android NDK / Clang	Android

A aplicação foi desenvolvida com janela base de 480×900 px, simulando um ecrã de smartphone em modo retrato. Esta escolha garante que a interface funciona adequadamente tanto em desktop como em mobile sem alterações de código ou layouts duplicados. A compilação CMake assegura que não existem dependências de plataforma no código de lógica; apenas o kit Qt é alterado entre targets.

Nota: As capturas de ecrã de Desktop e Mobile devem ser inseridas nesta secção após compilação e execução em ambiente real. As imagens comprovarão o correto funcionamento nos dois targets obrigatórios.

11. Dificuldades Encontradas e Soluções

Dificuldade	Solução Adotada
Sincronização animação QML / timer C++	Emissão de <code>dataChanged()</code> a cada passo (150 ms); o Behavior on x/y do QML interpola suavemente entre posições consecutivas
Múltiplas ligações de QTimer causavam execução duplicada	<code>disconnect()</code> explícito antes de cada <code>connect()</code> no <code>moveStepTimer</code> ; <code>Qt::UniqueConnection</code> no <code>boardingTimer</code>
Deteção de colisões em autocarros de múltiplas células	<code>isOccupied()</code> calcula todas as células ocupadas com base em direção e comprimento; <code>ignoreBusIndex</code> evita auto-colisão
Thread safety no carregamento assíncrono	Struct <code>LevelData</code> simples (sem objetos Qt não thread-safe) como contendor; consumida apenas na thread principal via <code>QFutureWatcher::finished()</code>
Remoção de autocarro cheio durante embarque animado	<code>clearSlots()</code> + <code>occupySlot()</code> recalculam os slots após cada remoção, mantendo consistência entre <code>m_parkedBuses</code> e <code>m_slots</code>
Imutabilidade em C++	Structs copiadas por valor (<code>BoardSnapshot</code>) simulam semântica imutável sem bibliotecas externas

12. Contribuição Individual

Componente	Gonçalo Marques (30012478)	Gonçalo Pardelhas (30014665)
Board, Bus (modelo de domínio)	Principal	Revisão e integração
GameController (lógica central)	Principal	Principal
GameAnalytics (paradigma funcional)	Principal	Revisão e testes
Persistence (JSON local)	Colaboração	Principal
Concorrência (QtConcurrent)	Colaboração	Principal
Interface QML e animações	Principal	Colaboração
Design dos níveis (JSON)	Colaboração	Principal
Testes unitários	Principal	Revisão
Relatório técnico	Colaboração	Colaboração

Ambos os elementos participaram ativamente em todas as fases do projeto, desde a definição da arquitetura até à validação final. A distribuição acima reflete a responsabilidade primária em cada componente, sendo que revisão, integração e decisões de arquitetura foram sempre tomadas em conjunto.
